

Combining Datasets: Merge and Join

Python Data Science Handbook — Chapter 3, Section 3.7

High-performance relational joins — the workhorse for stitching together tables that share keys.

1 Why we need joins at all

Background & Context

Real data almost never arrives in one tidy table. Instead it lives in several tables that each describe one “thing” and refer to each other through shared keys. You might have one table of employees (name, department, hire date), a separate table describing departments (name, manager, building), and a third holding monthly sales. Storing the data this way — each fact recorded exactly once, in the table where it belongs — is called *normalization*, and it is the design principle behind every relational database.

The price you pay for normalization is that, to answer an interesting question (“what is the average sales figure per building?”), you must first *recombine* the tables. The operation that does this is the **join**, and it comes straight out of *relational algebra*, the formal theory underneath SQL. Pandas gives you that same machinery on in-memory DataFrames through one extremely capable function, `pd.merge`, plus a convenience wrapper, `DataFrame.join`.

In the previous section we met `pd.concat`, which *stacks* tables — gluing them edge to edge along an axis. Concatenation is purely structural: it lines rows up by position or by index and asks no questions about the contents. A join is different in kind.

Intuition

Concat stacks; merge matches.

- `pd.concat` is like piling sheets of paper on top of one another (or laying them side by side). It cares about *shape*, not *meaning*.
- `pd.merge` is like a librarian cross-referencing two card catalogs: for each card in one, find the matching card in the other *by a shared value* (the *key*), and splice their information together. It cares about *content*.

If your two tables describe the *same rows* and you just want more columns or more rows of the same kind, reach for `concat`. If they describe *related but different* things linked by a common column, reach for `merge`.

2 A one-minute tour of relational algebra

Background & Context

Relational algebra, formulated by E. F. Codd in 1970, treats a table as a *relation*: a set of rows (tuples) over a fixed set of named columns (attributes). It defines a handful of operators that take relations in and produce relations out — selection (keep some rows), projection (keep some columns), union, intersection, and the *join*. SQL is essentially a friendly syntax over these

operators, and Pandas re-implements them for arrays in memory.

The **join** combines two relations by pairing rows whose key attributes agree. Everything else in this section — one-to-one, many-to-one, inner, outer — is just a special case or a policy choice layered on top of that single idea: *pair the rows whose keys match*.

3 pd.merge and the three cardinalities

The behavior of a join depends entirely on whether the key is *unique* on each side. There are three cases, and recognizing which one you are in is the single most important diagnostic skill for debugging a merge.

3.1 One-to-one

When the key is unique on *both* sides, each row in the left table matches at most one row on the right. The result simply lines the matching rows up and concatenates their columns — the row count cannot grow.

```
import pandas as pd

employees = pd.DataFrame({
    'name': ['Ada', 'Bhavin', 'Cleo', 'Dmitri'],
    'dept': ['Engineering', 'Sales', 'Engineering', 'Finance']
})
hires = pd.DataFrame({
    'name': ['Ada', 'Bhavin', 'Cleo', 'Dmitri'],
    'hire_year': [2019, 2021, 2018, 2022]
})
```

Python Refresher

Notice the deliberate typo: the `employees` table spells the second person Bhavin, but `hires` spells it Bhavin. (Look again — they differ.) Mismatched keys are the #1 source of “my merge lost rows” bugs. We will see the consequence in a moment; for now, assume the keys agree.

Assume both tables spell every name identically. Merging on the shared `name` column:

```
pd.merge(employees, hires)  # 'name' inferred as the key
```

	name	dept	hire_year
0	Ada	Engineering	2019
1	Bhavin	Sales	2021
2	Cleo	Engineering	2018
3	Dmitri	Finance	2022

Four rows in, four rows out: a clean one-to-one join. Because `name` is the only column the two frames share, `merge` infers it as the key automatically.

3.2 Many-to-one

Now suppose the key repeats on *one* side. Each department appears once in a department-info table, but many employees belong to the same department. Matching “many” employees against “one” department row copies that department’s information down onto every matching employee.

```
dept_info = pd.DataFrame({
    'dept': ['Engineering', 'Sales', 'Finance'],
    'building': ['B1', 'B2', 'B1']
})
pd.merge(employees, dept_info)  # key = 'dept', repeats on the left
```

	name	dept	building
0	Ada	Engineering	B1
1	Bhavin	Sales	B2
2	Cleo	Engineering	B1
3	Dmitri	Finance	B1

Key Idea

In a **many-to-one** join the result has the *same number of rows as the “many” side* (assuming every key finds a match). The unique side’s columns are *broadcast* down onto each repeated key — here Engineering appears twice on the left, so its building value B1 is duplicated into both rows. This is the most common join in practice: attaching reference/lookup information to a fact table.

3.3 Many-to-many

When the key repeats on *both* sides, the join produces a *cross-product within each matching key*: every left row with key k is paired with every right row with key k . This can make the result *larger* than either input.

```
skills = pd.DataFrame({
    'dept': ['Engineering', 'Engineering', 'Sales', 'Finance'],
    'skill': ['Python', 'SQL', 'Negotiation', 'Excel']
})
pd.merge(employees, skills)  # 'dept' repeats on BOTH sides
```

	name	dept	skill
0	Ada	Engineering	Python
1	Ada	Engineering	SQL
2	Cleo	Engineering	Python
3	Cleo	Engineering	SQL
4	Bhavin	Sales	Negotiation
5	Dmitri	Finance	Excel

Common Pitfall

Many-to-many joins can explode your row count. Two engineers $\times$ two engineering skills produced $2 \times 2 = 4$ rows for Engineering alone. If Engineering had 50 employees and 10 listed skills, that one key would yield 500 rows. Accidental many-to-many joins (usually because you forgot the key was non-unique on a side you assumed was unique) are a classic cause of mysteriously bloated DataFrames and out-of-memory errors. When in doubt, check `df['key'].is_unique` on each side *before* merging, or pass `validate='one_to_many'` to make Pandas raise if the cardinality is not what you expect.

4 Specifying the merge key

So far merge guessed the key. You will often need to be explicit.

4.1 The on keyword

When the frames share more than one column name, or you simply want to be unambiguous, name the key directly:

```
pd.merge(employees, dept_info, on='dept')
```

The `on` column must exist in *both* frames. You can also pass a list, `on=['key1', 'key2']`, to join on a composite key.

4.2 Differently-named columns: left_on and right_on

Sometimes the same concept is named differently in two tables — `name` here, `employee` there. Use `left_on` and `right_on` to point at each:

```
salaries = pd.DataFrame({
    'employee': ['Ada', 'Bhavin', 'Cleo', 'Dmitri'],
    'salary': [120, 95, 115, 130]
})
pd.merge(employees, salaries, left_on='name', right_on='employee')
```

	name	dept	employee	salary
0	Ada	Engineering	Ada	120
1	Bhavin	Sales	Bhavin	95
2	Cleo	Engineering	Cleo	115
3	Dmitri	Finance	Dmitri	130

Common Pitfall

This leaves you with *two* redundant columns, `name` and `employee`, holding identical information. Pandas does not drop either for you. Clean it up explicitly:

```
result.drop(columns='employee')
```

4.3 Joining on the index

If the key you want to join on is the *row index* rather than a column, set `left_index=True` and/or `right_index=True`:

```
emp_idx = employees.set_index('name')
sal_idx = salaries.set_index('employee')
pd.merge(emp_idx, sal_idx, left_index=True, right_index=True)
```

You can even mix the two styles — join a column on one side against the index on the other (`left_on='name', right_index=True`). Because index-based joins are so common, Pandas gives them a dedicated shortcut: `DataFrame.join` merges on indices by default.

```
emp_idx.join(sal_idx)  # equivalent to the merge above, by index
```

Intuition

`df.join(other)` is just `pd.merge` with `left_index=True`, `right_index=True` pre-filled, and (historically) a *left* join by default. Reach for `join` when both frames are already indexed by the key; reach for `pd.merge` when you need the full control of `on/left_on/right_on` or non-default join types.

5 Set arithmetic: the `how` parameter

This is the conceptual heart of the section. When some keys appear in one table but not the other, what should the join do with the unmatched rows? The `how` parameter answers exactly that, and it maps directly onto *set operations* on the two key columns.

Let two tables share the key `dept` but disagree on which departments they cover:

```
left = pd.DataFrame({'dept': ['Eng', 'Sales', 'Finance'],
                     'head': ['Ada', 'Bhavin', 'Cleo']})
right = pd.DataFrame({'dept': ['Eng', 'Sales', 'Legal'],
                      'floor': [3, 2, 5]})
```

The key sets are {Eng, Sales, Finance} on the left and {Eng, Sales, Legal} on the right. They share Eng and Sales; Finance is left-only; Legal is right-only. Each value of `how` keeps a different subset:

<code>how=</code>	Set operation on keys	Keys kept
<code>'inner'</code> (default)	intersection	Eng, Sales
<code>'outer'</code>	union	Eng, Sales, Finance, Legal
<code>'left'</code>	left set	Eng, Sales, Finance
<code>'right'</code>	right set	Eng, Sales, Legal

Wherever a kept key has no partner on the other side, Pandas fills the missing columns with NaN.

5.1 The four joins, visualized

The diagram below shows the two key sets and, for each how, which rows survive and where NaN appears. Read each column as “what the result table looks like.”

	inner	left	right	outer
Eng	match	match	match	match
Sales	match	match	match	match
Finance	—	NaN →	—	NaN →
Legal	—	—	← NaN	← NaN

row kept, both sides present
row kept, other side filled with NaN
row dropped from result

Concretely, the *outer* join — the union, which keeps everything — looks like this:

```
pd.merge(left, right, how='outer')
```

```

      dept  head  floor
0      Eng   Ada   3.0
1    Sales Bhavin  2.0
2  Finance  Cleo   NaN
3    Legal   NaN   5.0

```

Key Idea

Two consequences of NaN-filling are worth internalizing. First, the `floor` column came in as integers but prints as `3.0` — introducing NaN forced the column to *float*, because classic NumPy integer arrays have no missing value. (Pandas’ newer nullable `Int64` dtype avoids this, but the default does not.) Second, an *inner* join silently *discards* unmatched rows. If your merged frame has fewer rows than you expected, an unintended inner join on a mismatched key is the usual culprit — which loops right back to the Bhavin/Bhavin typo from earlier: a misspelled key simply fails to match and vanishes under the default inner join.

6 Overlapping non-key columns: suffixes

What if both frames carry a column of the same name that is *not* the key? Pandas cannot keep both under one name, so it disambiguates by appending suffixes — `_x` for the left, `_y` for the right by default.

```

q1 = pd.DataFrame({'dept': ['Eng', 'Sales'], 'rank': [1, 2]})
q2 = pd.DataFrame({'dept': ['Eng', 'Sales'], 'rank': [2, 1]})
pd.merge(q1, q2, on='dept')

```

	dept	rank_x	rank_y
0	Eng	1	2
1	Sales	2	1

The cryptic `_x/_y` are rarely what you want in a report. Override them with meaningful labels:

```
pd.merge(q1, q2, on='dept', suffixes=('_q1', '_q2'))
```

	dept	rank_q1	rank_q2
0	Eng	1	2
1	Sales	2	1

7 Mini case study: states × population × area

Let us put every piece together on three small, original tables: a list of states, their populations, and their land areas. We want one tidy frame and then a derived quantity, population density.

```
states = pd.DataFrame({
    'state': ['California', 'Texas', 'Florida', 'Alaska'],
    'abbrev': ['CA', 'TX', 'FL', 'AK']
})
population = pd.DataFrame({
    'state': ['California', 'Texas', 'Florida', 'Alaska'],
    'pop_millions': [39.0, 30.0, 22.0, 0.73]
})
area = pd.DataFrame({
    'state': ['California', 'Texas', 'Florida'], # Alaska missing!
    'area_kkm2': [424.0, 696.0, 170.0]
})
```

First, a clean many-to-one chain joining states to their populations (one-to-one here), then to area. We deliberately left Alaska out of the area table to exercise the missing-key behavior:

```
merged = (states
    .merge(population, on='state') # one-to-one
    .merge(area, on='state', how='left')) # keep all states
```

	state	abbrev	pop_millions	area_kkm2
0	California	CA	39.00	424.0
1	Texas	TX	30.00	696.0
2	Florida	FL	22.00	170.0
3	Alaska	AK	0.73	NaN

Using `how='left'` guarantees every state survives even though Alaska has no area row; its `area_kkm2` is `NaN`. Now a derived column, density (people per km^2), computed only where area is known:

```
merged['density'] = merged['pop_millions'] * 1e6 / (merged['area_kkm2'] * 1
```

```
e3)
merged[['state', 'density']]
```

	state	density
0	California	91.981132
1	Texas	43.103448
2	Florida	129.41176
3	Alaska	NaN

Intuition

Notice how the `NaN` *propagates* cleanly through the arithmetic — Alaska’s missing area yields a missing density rather than crashing the computation. This is the payoff of Pandas’ labeled, missing-aware design from §3.1: joins introduce `NaN` where data is absent, and downstream vectorized math simply carries it along, so partial data never derails the whole pipeline.

The chained `.merge(...).merge(...)` style reads top to bottom like a recipe and is the idiom you will reach for constantly: start from the “spine” table you want one row per, then left-join each source of additional columns onto it.

Section recap

- A **join** pairs rows from two tables by a shared **key**; it comes from relational algebra and is the content-aware counterpart to the purely structural `pd.concat`.
- **Cardinality** drives behavior: *one-to-one* lines rows up; *many-to-one* broadcasts the unique side onto repeats; *many-to-many* forms a per-key cross-product and can blow up row counts.
- **Specifying the key**: `merge` infers shared columns by default; use `on=`, or `left_on/right_on` for differently-named columns (then drop the redundant copy), or `left_index/right_index` (and the `join` shortcut) for index-based joins.
- **how=** chooses the set operation on keys: `inner` (intersection, default), `outer` (union), `left`, `right`. Unmatched rows are `NaN`-filled, which can promote integer columns to float; a stray inner join silently drops unmatched rows.
- Overlapping non-key columns get `suffixes=('_x', '_y')`; rename them for readable output.